

ComputerAgent Cross-Platform Validation and Extension Report

Executive summary

The attached 32-page report is **directionally correct and technically viable**, and the additional platform research strengthens rather than overturns its central architecture: ComputerAgent should own the controller, durable state, policy, verification, recovery, and routing, while commodity mechanisms handle browsers, accessibility, vision, inference, and isolation. The report's execution hierarchy—API/tool → browser semantics → desktop accessibility → visual grounding → isolated desktop → human handoff—remains the right foundation. Its bounded-context approach for long-running work also remains appropriate.  

The major change is that the design must stop treating a 12 GB desktop RTX 4070 as the product envelope and instead treat **hardware capability as runtime state**. This matters even within the same GPU name: NVIDIA specifies 12 GB for the desktop RTX 4070 but only 8 GB for the RTX 4070 Laptop GPU. Apple Silicon has a different memory model again. ComputerAgent therefore needs a `HardwareProfile` → `InferenceProfile` decision layer from the beginning, rather than choosing one model globally. ¹

The second major change concerns the “second cursor.” On Windows, the report's assumption is strongly validated: current Microsoft UI Automation tooling distinguishes semantic operations such as `invoke`, `set-value`, inspection and semantic scrolling—which can work without normal pointer injection—from physical click/keyboard operations that require an interactive foreground desktop. On macOS, `AXUIElement` provides analogous semantic actions and settable attributes, but Apple does **not** provide a blanket guarantee that arbitrary applications will accept all actions without activation, focus changes, or incomplete accessibility implementations. macOS should therefore be P0, but its background-control guarantees must be discovered and recorded per application/action rather than assumed. ²

The third correction is important for a consumer product: **a full VM cannot be the universal fallback**. Apple's Virtualization framework can run macOS and Linux VMs, while Windows Sandbox provides a disposable virtualized Windows environment, but Windows Sandbox is unavailable on Home editions and itself consumes substantial memory; Apple VMs similarly compete with the local model and normal applications for memory. The low-hardware product path therefore needs to be `semantic host` → `browser-specific background path` → `visual/background path where proven` → `optional isolated desktop on capable machines` → `handoff`, not “everything unsupported goes to a VM.” ³

Bottom-line recommendation: proceed. ComputerAgent is buildable as a local-first consumer product on macOS and Windows, with Linux as P1, provided V1 does **not** promise arbitrary background manipulation of every application, identical intelligence on every laptop, or zero-user-interaction installation/authentication. The correct promise is closer to:

ComputerAgent works locally, chooses the safest interaction path supported by the device and application, lets the user keep working whenever a proven background route exists, and hands control back cleanly when the operating system, application, authentication boundary, or hardware makes autonomous operation unsafe.

The architecture should be frozen only after the Phase 0 gates later in this report pass.

Platform feasibility and non-interference

macOS

macOS is viable as a P0 platform. `AXUIElement` exposes application and UI-element objects, attributes, settable values, and actions such as accessibility actions through APIs including `AXUIElementPerformAction`. It also explicitly defines failure cases such as unsupported operations, incomplete application accessibility implementations, disabled accessibility APIs, invalid elements, and operations the target application cannot complete. That error surface is exactly why ComputerAgent should model accessibility as a capability adapter rather than assume a universal desktop API. ⁴

This means the report's Windows-oriented concept:

```
inspect element
→ invoke / set value / semantic scroll
→ observe state change
→ verify
```

can be reproduced on macOS in principle. What cannot be assumed is that every target application will preserve foreground focus while doing so. The correct ComputerAgent abstraction is therefore:

```
MacAccessibilityAdapter
  inspect()
  capabilities(element)
  execute_semantic(action)
  observe_focus_delta()
  observe_window_delta()
  classify_result()
```

Each action receipt should include not only success/failure but whether the target application came to the foreground, whether keyboard focus changed, whether a new window appeared, and whether a previously bound AX element became stale.

The user must explicitly grant a third-party app Accessibility access in macOS Privacy & Security before that app can control the Mac through accessibility features. This is not a permission ComputerAgent can silently grant itself. Apple similarly requires user authorization for screen/system-audio capture; ScreenCaptureKit is the modern framework for capturing displays, applications and windows. ⁵

That creates a hard product requirement:

```
First launch
↓
Explain why Accessibility is needed
↓
User grants Accessibility
↓
Explain why Screen Recording is needed
↓
User grants Screen Recording
↓
ComputerAgent verifies both
↓
Run 10-second capability test
↓
Ready
```

Permission onboarding is therefore part of P0 engineering, not a later UI detail.

Native macOS applications will generally be the best target for AX-based semantics. Chromium gives ComputerAgent a better route through the browser layer. Electron is more variable: Electron exposes Chromium's accessibility architecture to assistive technologies, but its documentation notes that generating the accessibility tree has a performance cost and accessibility behavior can depend on how that application/framework is configured. ⁶

The practical routing order on macOS should consequently be:

Target	Preferred P0 route	Fallback
Web content in agent-managed Chromium	Playwright/CDP semantics	screenshot grounder
Native macOS application	AXUIElement	visual route/handoff
Electron app	AX tree if sufficient	browser/Electron-specific route, then vision
Custom-rendered app	whatever AX exposes	vision
Password/security/system consent	human	no autonomous bypass
Coordinate-only interaction	isolated workspace if available	foreground handoff

Playwright is particularly valuable because locators are re-resolved for each action, auto-wait for actionability, and can use accessibility-oriented locators such as roles and labels. Playwright also states that a `Page` does not need to be brought to the front before interacting with it. Attaching to arbitrary existing

Chromium sessions over CDP is possible, but Playwright documents CDP attachment as lower fidelity than its normal Playwright protocol, so an **agent-managed browser profile should be the default P0 browser**; attaching to the user's existing browser should be an explicitly tested secondary mode. ⁷

There is additional 2026 evidence for this architecture from Cua Driver. Its cross-platform driver defaults to background delivery, exposes accessibility and pixel actions, browser automation, screenshots, action verification and session recording, and only escalates after a background route has failed or refused. Its own macOS/Chromium documentation records cases where some trusted pointer routes can activate a standalone Chromium window, while page-aware CDP operations can target an occluded page without moving the real cursor. That is almost exactly the distinction ComputerAgent needs to encode. ⁸

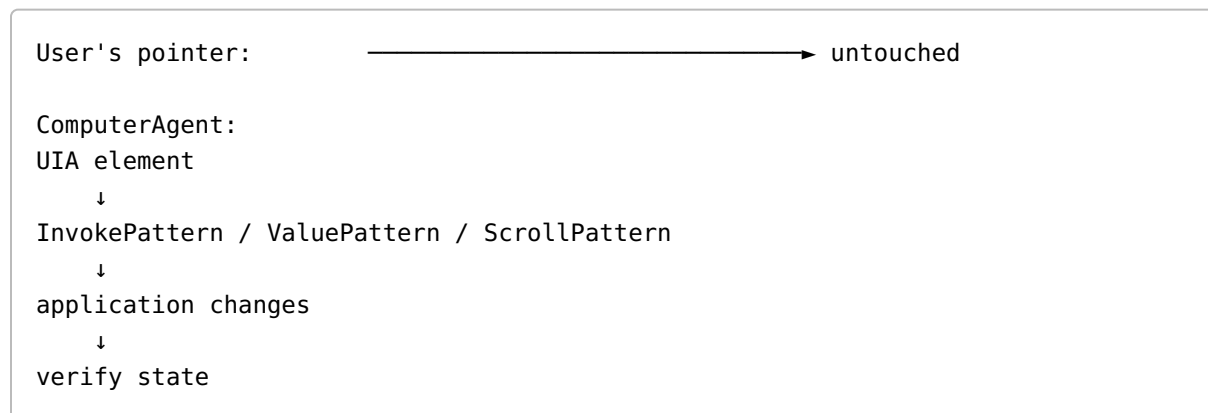
Recommendation: classify actions as `BACKGROUND_PROVEN`, `BACKGROUND_BEST_EFFORT`, or `FOREGROUND_REQUIRED`. Do not expose “macOS background control” as a single Boolean.

Windows

Windows is the stronger P0 semantic-control environment.

Microsoft's current WinApp UI Automation tooling explicitly separates semantic UIA operations from physical input. `inspect`, `search`, `get-property`, `get-value`, `wait-for`, `set-value`, `invoke`, accessibility scrolling and normal screenshot operations can use UI Automation patterns, while mouse clicks, drag, wheel and `SendInput`-style keyboard operations require an unlocked interactive desktop and generally a foreground target. Microsoft explicitly recommends the UIA-pattern verbs when real input is unnecessary. ⁹

That directly validates the report's non-interference design:



There are still significant boundaries. Integrity-level isolation can prevent a normally privileged automation process from controlling an elevated application. Windows supports a special UIAccess model, but Microsoft requires trusted signing and protected installation locations for such applications, and enabling cross-integrity automation expands the security surface. UAC secure-desktop behavior is also intentionally restrictive. ¹⁰

For a consumer ComputerAgent, **do not make UIAccess elevation a V1 requirement**. A UAC/security boundary should ordinarily become a handoff:

“ComputerAgent needs you to approve this Windows security prompt. I’ll continue afterward.”

That is safer than training users to accept an AI application that attempts to weaken secure-desktop behavior.

Browser operation should remain Playwright-first on Windows as well. This provides a route that generally does not need the host physical pointer and isolates ComputerAgent from many UIA framework inconsistencies inside Chromium. ¹¹

Cross-platform conclusion

Capability	macOS P0	Windows P0	Linux P1	Product promise
Browser DOM interaction without real cursor	Yes	Yes	Yes	Strong
Semantic native-app interaction	Yes, app-dependent	Yes, relatively strong	AT-SPI-dependent	Supported, not universal
Arbitrary GUI background coordinate input	No universal guarantee	No universal guarantee	No universal guarantee	Do not promise
Screenshot capture	Yes, permission required	Yes	compositor-dependent	Supported
Security/UAC/TCC dialogs fully autonomous	No	No	varies	Human handoff
User keeps working during semantic route	Target and test	Strongly feasible	test P1	Core feature
User keeps working during arbitrary host pixel control	Unreliable	Unreliable	Unreliable	Isolate or handoff

Cua Driver's current Linux design also requires its daemon to participate in the graphical session and AT-SPI session bus, reinforcing the attached report's decision to keep Linux as a separate adapter rather than treating synthetic input as a universal answer. ¹²

Local models and hardware-adaptive inference

The report's conclusion that ComputerAgent should **not train a foundation model from scratch** remains correct. The model stack available by September 2026 is significantly more favorable to this project than even the report's 4070 framing suggests.

Qwen's official current repository says Qwen3.5 introduced a unified vision-language foundation, and the small Qwen3.5-9B, 4B, 2B and 0.8B models were released in March 2026. Importantly for a consumer

application, Qwen explicitly documents Qwen3.5 text-and-vision support in `llama.cpp`, plus `mlx-lm` / `mlx-vlm` support on Apple Silicon. ¹³

UI-TARS-1.5-7B remains the strongest specialized visual candidate from the baseline. ByteDance's official model card labels the checkpoint Apache-2.0, identifies it as a Qwen2.5-VL-derived multimodal GUI model, and documents local inference through Transformers as well as vLLM/SGLang paths. ¹⁴

A new candidate should be added to the report's P0 benchmark: **Gemma 4 E2B and E4B**. Google released Gemma 4 in March 2026 and describes E2B/E4B as multimodal edge-oriented models targeting environments from mobile devices through laptops; its official model card lists Apache 2.0 licensing. This does not mean Gemma will beat Qwen at GUI work—it has not been selected for that—but it is a worthwhile low-hardware planner/VLM control candidate. ¹⁵

Recommended model race

Candidate	ComputerAgent role	Priority
Qwen3.5-2B	low-memory planner/router + emergency lightweight VLM	P0
Qwen3.5-4B	likely default consumer planner/multimodal model	P0 highest
Qwen3.5-9B	higher-capability local profile	P0 highest
UI-TARS-1.5-7B	dedicated screenshot grounding/GUI executor	P0 highest
Gemma 4 E2B/E4B	low-memory independent comparator	P0
Qwen3-8B	regression/control from BrowserAgent	P0 control
Llama 3.2 11B Vision	baseline comparator from the attached report	P1/control
Qwen3.8-27B	rare high-memory/slow escalation planner	P1 research

Qwen3.8 is specifically marketed by Qwen as improving long-horizon agentic execution, but 27B dense-class storage alone makes it the wrong universal laptop model. It belongs in an optional high-memory escalation profile, not the interaction loop. ¹³

Runtime strategy

The report's recommendation of `llama.cpp` as the primary runtime becomes **stronger** when the product is expanded to Macs and ordinary Windows laptops. `llama.cpp` supports Apple Silicon through Metal, NVIDIA through CUDA, other GPUs through Vulkan/SYCL paths, aggressive low-bit quantization, and CPU+GPU hybrid execution. ¹⁶

For Apple Silicon, **benchmark MLX alongside llama.cpp**. Qwen officially supports its small Qwen3.5 line through MLX, and Apple's MLX-LM benchmark demonstrates why that path deserves serious evaluation: on a 64 GB M4 Max, a quantized Qwen3 4B model used approximately 3.35 GB at 4-bit and generated about 134.5 tokens/s in that specific benchmark. That is **not** an M2 result and not a Qwen3.5 benchmark, so it must not be projected literally onto consumer Macs; it merely demonstrates that low-bit 4B inference can be very fast and compact on higher-end Apple Silicon. ¹⁷

`vLLM` should be downgraded from “possible production runtime” to **benchmark/server comparator** for Windows consumer builds: upstream vLLM explicitly states that native Windows is not supported and points users to WSL or community forks. On Apple Silicon, the newer community-maintained `vllm-metal` plugin now uses MLX and supports macOS 15+ Apple Silicon, making it worth benchmarking—but its extra Python/service layer is unnecessary unless it beats embedded alternatives enough to justify packaging complexity.

18

Ollama remains an excellent developer and advanced-user integration. It supports Metal on Apple devices, CUDA on RTX 40-series cards including the 4060/4070, and Vulkan paths on Windows/Linux. For the consumer product, however, requiring people to install/manage Ollama separately would violate the desired “install ComputerAgent and it works” experience. The production build should preferably embed or own its inference process, while an `OllamaAdapter` remains available for development and advanced settings.

19

Hardware profiles

The following is the recommended **starting hypothesis**, not claimed benchmark data. The throughput ranges are engineering envelopes to validate in Phase 0; where no same-model/same-device primary benchmark exists, exact performance remains unspecified. Q4/Q5 should be tested against task accuracy before shipping.

Hardware tier	Recommended default	Runtime	Model-memory budget	Expected planning envelope*	Visual path	Shipping profile
8 GB M2/M-series Mac	Qwen3.5-2B Q4; Gemma 4 E2B comparator	llama.cpp Metal; MLX comparator	~2.5–3.5 GB	~15–40 tok/s hypothesis; semantic decision 2–4 s	same small VLM; no resident UI-TARS	Lite
16 GB Apple Silicon	Qwen3.5-4B Q4/Q5	llama.cpp Metal vs MLX	~3.5–5.5 GB	~30–70 tok/s hypothesis; semantic 1–3 s	UI-TARS loaded on demand if memory test passes	Standard
24 GB Apple Silicon	Qwen3.5-9B Q4 or 4B planner	llama.cpp/ MLX	~5–8 GB for main model	~20–50 tok/s for 9B hypothesis; semantic 1–3 s	UI-TARS 7B on demand; possibly cached	Enhanced

Hardware tier	Recommended default	Runtime	Model-memory budget	Expected planning envelope*	Visual path	Shipping profile
Windows 16 GB + integrated GPU	Qwen3.5-2B or 4B Q4	llama.cpp Vulkan/CPU hybrid	~3-4.5 GB shared/system	~8-30 tok/s hypothesis; semantic 2-6 s	lightweight VLM only; semantics-first	Lite
RTX 4060/4070 Laptop, 8 GB VRAM	Qwen3.5-4B Q4/Q5	llama.cpp CUDA	≤~6.5 GB total accelerator usage	~40-80 tok/s hypothesis; semantic <2 s target	UI-TARS swapped/on demand	Standard GPU
Desktop RTX 4070, 12 GB VRAM	Qwen3.5-9B Q4 or 4B planner	llama.cpp CUDA	≤~9.5-10 GB	~50-100 tok/s hypothesis; semantic <2 s target	UI-TARS on demand; <4 s visual target	Enhanced GPU

*These are deliberately conservative **Phase 0 engineering hypotheses**, not published model/device benchmarks. They exist so the build specification has something testable. The shipping profiler should replace them with measured local calibration during installation.

A critical correction to the original report is visible in the last two rows. A **desktop RTX 4070 has 12 GB**, but NVIDIA specifies the RTX 4060 Laptop and RTX 4070 Laptop GPUs at **8 GB**. ComputerAgent must therefore probe actual memory rather than infer capability from the label “4070.” ²⁰

For Apple Silicon, similarly, the relevant resource should be represented internally as **available unified-memory headroom**, not a hard-coded “VRAM” field.

A suitable hardware profiler contract is:

```
HardwareProfile
  os
  os_version
  architecture

  physical_memory
  available_memory

  gpu_vendor
  gpu_model
  dedicated_vram?      # null/NA on unified-memory systems
  unified_memory?
```

```
accelerator_backend      # Metal / CUDA / Vulkan / CPU
```

```
thermal_class
```

```
battery_state
```

```
virtualization_available
```

```
screen_capture_available
```

```
accessibility_available
```

↓

InferenceProfile

```
planner_model
```

```
planner_quant
```

```
vision_model
```

```
vision_quant
```

```
runtime
```

```
max_context
```

```
max_image_resolution
```

```
cache_budget
```

```
model_residency_policy
```

This is an important extension to the attached report: **model selection is part of system scheduling, not a Settings dropdown.**

Isolation, packaging, and consumer distribution

Isolated desktop strategy

The report is right that coordinate-based interaction needs an isolated environment when host interaction would fight the user, but a consumer product needs more nuance.

Apple's Virtualization framework provides APIs for macOS and Linux guests and supports virtual pointing/keyboard devices. macOS guests on Apple Silicon use Apple's macOS restore images and run the same architecture as the host. ²¹

Windows Sandbox provides a disposable hardware-virtualized Windows desktop, supports configurable memory and vGPU behavior, and starts with a default maximum allocation around 4 GB. However, Microsoft limits Windows Sandbox to Pro, Enterprise and Education-class editions; it is not supported on Windows Home. ²²

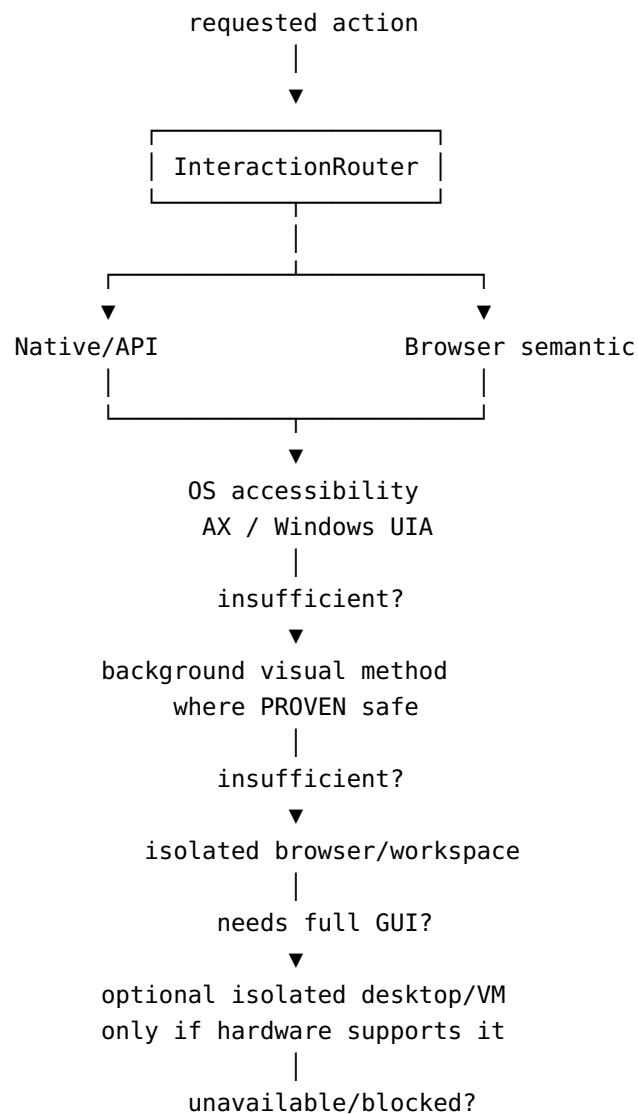
Those facts make the original “VM fallback” too broad for the final product.

On an 8 GB Mac, running:

macOS
+ user's normal apps
+ ComputerAgent UI
+ 2B/4B model
+ screenshot pipeline
+ another graphical OS

is not a sensible universal V1 requirement. That conclusion is an engineering inference from the independent memory requirements rather than a platform prohibition. ²³

The revised escalation ladder should therefore be:



▼
human handoff

This also exposes a product truth that should go into `BUILD_SPEC.md`: **an isolated VM is not automatically equivalent to the user's host desktop**. The user's installed native apps, active logins, local files and application state may not exist in the guest. The VM is excellent for isolated browser work and provisioned workflows; it is not a magical copy of every host application.

Cua's current open-source infrastructure is worth auditing precisely here. Its current stack includes cross-platform computer-use drivers plus agent-ready isolated desktop infrastructure, and its driver already implements an escalation distinction between accessibility/background delivery and foreground interaction.

24

macOS distribution

For software distributed outside the Mac App Store, Apple expects Developer ID signing so Gatekeeper can identify the publisher, and Apple supports notarizing Developer-ID-signed applications and installer artifacts. Apple's notarization process works with distributable formats such as application archives/packages and DMGs and is designed to be part of an automated release pipeline. 25

The production Mac flow should therefore be:

```
Build universal/arm64 app
  ↓
Sign main app
  ↓
Sign every nested helper/runtime
  ↓
Enable hardened runtime
  ↓
Create DMG/PKG
  ↓
Submit for notarization
  ↓
Verify notarization
  ↓
Staple ticket where applicable
  ↓
Release
```

Notarization does **not** eliminate Accessibility or Screen Recording consent. Those remain explicit user permissions. 5

Consequently, a build that tells users to run:

```
sudo ...  
xattr -dr ...  
disable Gatekeeper ...
```

should be considered a **release-blocking failure**. That may be acceptable during development, but it cannot be part of consumer onboarding.

For P0 I recommend direct Developer ID distribution rather than making Mac App Store acceptance a dependency. Apple requires App Sandbox for Mac App Store distribution while outside-store Developer ID apps can use different capability choices; ComputerAgent's unusually broad accessibility role deserves a separate App Store feasibility/legal review rather than being assumed compatible. ²⁶

Windows distribution

Microsoft now offers two especially relevant consumer routes. MSIX submitted through the Microsoft Store is re-signed by Microsoft and receives Store-managed distribution/update behavior. For direct distribution, Microsoft supports signed MSIX packages with `.appinstaller` metadata for automatic updates, and recommends trusted code-signing mechanisms such as Azure Artifact Signing or traditional CA-issued certificates. ²⁷

SmartScreen is not solved simply by “having a signature.” Microsoft documents reputation as a separate consideration for newly distributed direct-download applications, while self-signed certificates are unsuitable for public consumer distribution. ²⁸

For ComputerAgent I would test both:

```
Route A:  
Microsoft Store MSIX  
→ cleaner consumer trust/update path  
  
Route B:  
Direct signed MSIX + .appinstaller  
→ more distribution control  
→ Windows automatic update support
```

Microsoft's App Installer can automatically check for and apply updates for direct-distributed MSIX applications when configured through an `.appinstaller` manifest. ²⁹

The installer decision must be validated against ComputerAgent's accessibility, helper-process and local-model requirements before freezing it.

Model installation and updates

Do **not** put every supported model into the main installer.

A consumer-ready Model Manager should instead:

```
hardware detection
  ↓
choose recommended profile
  ↓
show download size
  ↓
check disk / memory requirements
  ↓
download resumably
  ↓
verify cryptographic hash
  ↓
activate atomic model version
  ↓
run calibration test
  ↓
keep previous known-good model metadata
```

The release manifest should define:

```
model_id
source
license
version
sha256
file_size
architecture
quantization
supported_runtimes
minimum_memory
recommended_memory
vision_support
benchmark_revision
rollback_target
```

This is also where model-license auditing belongs. Qwen's current project explicitly tells users to consult the license attached to each released model weight rather than assuming one global family license; UI-TARS-1.5-7B currently reports Apache-2.0, while Gemma 4 reports Apache-2.0. ³⁰

OSS landscape and architectural implications

The most important external finding is that by September 2026 several independent projects are converging on pieces of the architecture proposed in the attached report. This lowers technical risk but also

means ComputerAgent should reuse more of the execution layer and focus differentiation on reliability, local adaptivity, safe autonomy and consumer UX.

Project	What it validates	ComputerAgent use
Cua / Cua Driver	macOS/Windows/Linux driver, accessibility + pixels, background/foreground delivery, browser targeting, snapshot/action/verify concepts	P0 source/reuse audit
Trope CUA	dedicated Windows/macOS background-computer-use layer with accessibility trees, screenshots and action receipts	P0 comparison
UI-TARS Desktop / Agent TARS	cross-platform GUI agent UI, local GUI model, hybrid GUI/DOM browser strategy, event-stream-driven agent interface	P0 architecture/UI reference
Agent S	mature cross-platform GUI-agent orchestration and grounding separation	evaluation/reference, not core controller
OmniParser	screenshot → structured interactable-element representation	grounding benchmark, license audit
Browser Use	mature browser-agent harness and local-model integration	browser implementation reference
OSWorld/ BrowserGym	reproducible computer/browser evaluation	mandatory benchmark layer

Cua Driver is the closest new evidence to the report's “interaction fabric” concept. Its documentation explicitly describes native macOS, Windows and Linux operation; exact window targeting; accessibility and pixel actions; background/foreground delivery; typed browser automation; trajectory recording; and a default preference for background delivery before foreground escalation. ¹²

Trope CUA independently describes its goal as background computer use on Windows and macOS while avoiding accidental movement of the user's real cursor or changes to their active application when the target surface permits it. That is useful corroboration that the “semantic second cursor” is an implementable engineering direction rather than merely a theoretical construct. ³¹

ByteDance's UI-TARS Desktop/Agent TARS currently implements both native GUI-agent operation and a hybrid browser agent capable of GUI, DOM or hybrid browser strategies. Its event-stream architecture is also worth studying for ComputerAgent's user-facing live activity view. ³²

Agent S is valuable for a different reason. The project now supports macOS, Windows and Linux and reports very strong recent OSWorld performance, but its recommended setup still pairs a large cloud reasoning model with UI-TARS grounding. Therefore its results **do not prove that a 2B–9B fully local planner can match frontier end-to-end intelligence**. They do demonstrate that specialized grounding plus structured orchestration continues to be competitive. ³³

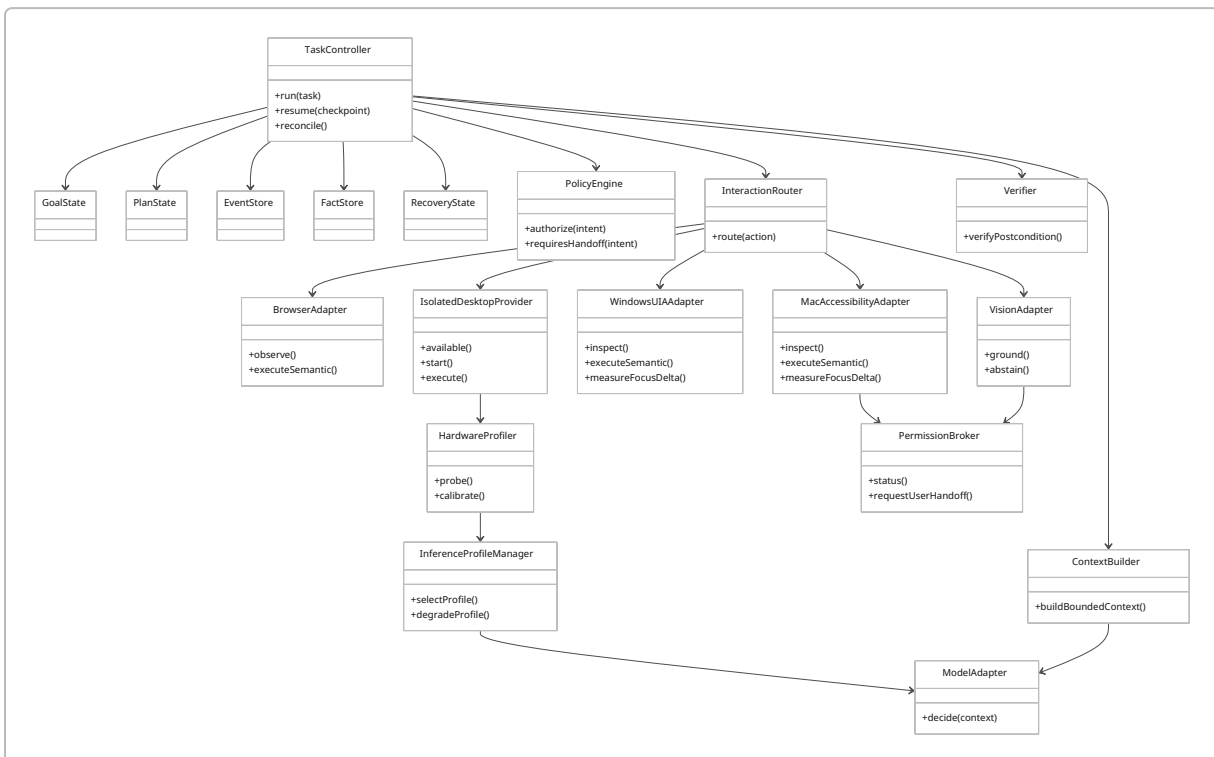
That distinction is important. ComputerAgent's defensible product advantage should not be:

“We invented screenshot clicking.”

It should be:

“We built a trustworthy local task operating system around whatever models and interaction mechanisms are currently best.”

Adapter relationship



This extends the original report's adapter model with four components that should become explicit V1 architecture: **HardwareProfiler**, **InferenceProfileManager**, **PermissionBroker**, and **per-action background-capability metadata**.

Phase-zero gates and build plan

What must be experimentally proven

These should become hard gates in **BUILD_SPEC.md**, not aspirational tests.

Experiment	Pass condition	Fail consequence
Clean macOS installation	Signed/notarized build launches through normal consumer flow; Accessibility + Screen Recording are granted through documented OS UI; no Terminal/Gatekeeper workaround	Packaging architecture not ready
Clean Windows installation	Valid trusted signature/package; normal install/update path; application starts without requiring dev tools	Distribution architecture not ready
macOS semantic non-interference	500 AX/Playwright semantic mutations cause zero movement of physical pointer ; every operation tagged <code>BACKGROUND_PROVEN</code> leaves the user's active app/focus unchanged	offending action/app loses background-safe status
Windows semantic non-interference	500 UIA/Playwright semantic mutations cause zero physical-pointer movement and no unexpected foreground activation	route downgraded/fallback required
Browser background test	500 Playwright actions while user works elsewhere; zero host-pointer movement, zero unrelated focus theft, correct postconditions	browser route redesign
Native/Electron coverage	Fixture suite records capability support across native, Electron and custom UI; unsupported controls route automatically rather than blindly injecting input	InteractionRouter fails
Host-coordinate negative control	Demonstrate that generic coordinate injection can interfere; prove it is never selected by default	architecture violation
Local model schema	$\geq 99.5\%$ valid structured decision outputs	model/profile rejected
Easy semantic action selection	$\geq 90\%$ accepted-action accuracy on fixed fixture set, preserving baseline report criterion	model/profile rejected
Low-tier endurance	1-hour run with zero OOM/crash and acceptable OS memory pressure	smaller model/context required
Standard-tier latency	semantic decision $p95 \leq 2$ s preferred; visual decision $p95 \leq 4-5$ s	optimize/model downgrade
Lite-tier latency	semantic $p95 \leq 4$ s and visual $p95 \leq 10$ s	classify device unsupported for full visual mode
Grounding safety	zero destructive false-positive clicks in destructive-control fixture; uncertain cases abstain	visual stack rejected

Experiment	Pass condition	Fail consequence
Isolated desktop	On supported hardware, 100 coordinate actions cause zero host pointer/focus effect; clean teardown and recovery	VM provider rejected
Handoff	TCC, UAC, login, MFA, CAPTCHA and consequential-action fixtures pause before unauthorized action and resume from saved state	release blocker
Crash reconciliation	Every fault-injection point around state changes avoids duplicate side effects	release blocker
Long horizon	Prompt size remains approximately flat from 200→1,000 actions while constraints/facts remain recoverable	release blocker for core product

The macOS permission gate is mandatory because Apple explicitly requires the user to authorize Accessibility control. Windows secure-desktop/input restrictions likewise make some interaction boundaries inherently non-autonomous. ³⁴

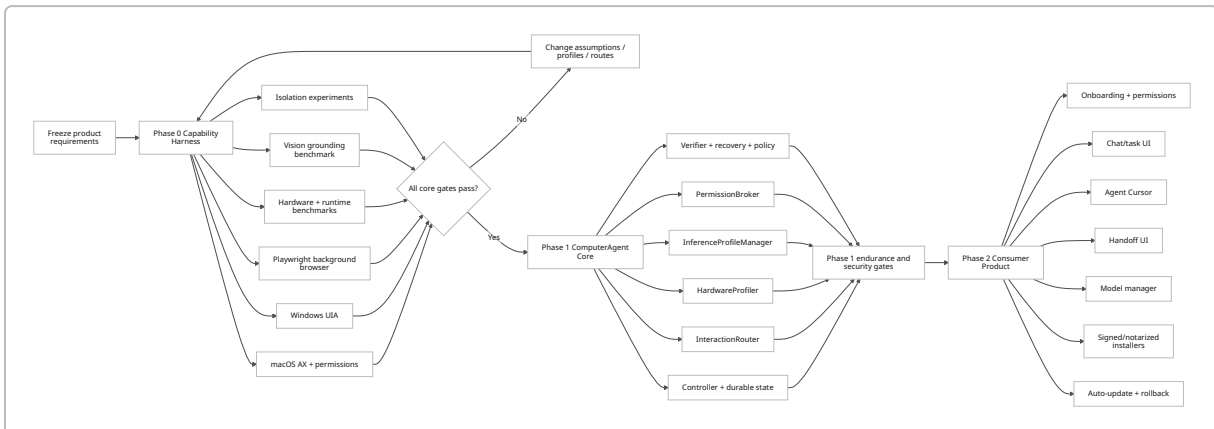
Application matrix for P0

Do not test only Chrome and declare desktop automation solved. The minimum matrix should include:

Class	macOS examples	Windows examples	What is being tested
Browser	Chrome/Chromium	Chrome/Edge	Playwright/background semantics
Native simple	TextEdit/Finder	Notepad/File Explorer	basic accessibility patterns
Native complex	Calendar/system app fixture	modern WinUI/WPF app	tree depth/state change
Electron	VS Code + one second app	VS Code + one second app	accessibility-tree quality
Custom rendered	selected test app	selected test app	visual escalation
Security boundary	Privacy & Security prompt	UAC/credential prompt	mandatory handoff

Electron and custom rendering deserve dedicated rows because accessibility coverage is not uniform across UI frameworks, and Microsoft itself documents variable coverage for Electron/custom frameworks compared with traditional Windows UI frameworks. ⁹

Build plan



Phase 0 — evidence, not product polish

Create a standalone capability/benchmark harness first. It should measure physical cursor position, foreground application, keyboard focus, target window, accessibility action, model latency, memory, before/after screenshots, postconditions and hardware characteristics. Run Mac and Windows experiments from the same fixture definitions.

At the end of Phase 0, `BUILD_SPEC.md` should contain measured answers for:

```
Which Mac actions are truly background-safe?
Which Windows actions are truly background-safe?
Which applications require escalation?
Which model is default on each hardware tier?
Which runtime wins each tier?
Can UI-TARS fit/usefully run on each tier?
Which machines can support an isolated desktop?
What is the actual latency budget?
```

Until those are answered, **do not let Claude build a giant production UI.**

Phase 1 — ComputerAgent core

Build the OS-independent controller and adapters around the measured results:

```
TaskController
GoalState
PlanState
EventStore
FactStore
ArtifactStore
```

```
RecoveryState

ContextBuilder
PolicyEngine
Verifier

HardwareProfiler
InferenceProfileManager
ModelManager
PermissionBroker

InteractionRouter
  BrowserAdapter
  AccessibilityAdapter
    MacAXAdapter
    WindowsUIAAdapter
  VisionAdapter
  IsolatedDesktopProvider
```

The core should never contain:

```
if gpu_name == "RTX 4070":
    model = ...
```

It should consume measured capabilities.

Likewise, the local model should never decide:

```
"I think this computer probably lets me background-click."
```

The route must come from capability state established by deterministic probes and adapters.

Phase 2 — UI and distribution

Only after the controller can complete unseen fixture tasks should the consumer experience be finished:

```
Download
↓
Install
↓
Permissions
↓
Hardware calibration
↓
```

```
Recommended model download
↓
First test
↓
Task box
↓
short visible plan
↓
agent works
↓
optional Agent Cursor
↓
handoff only when required
↓
verified completion
```

The UI should conceal technical implementation details by default. Users should see:

ComputerAgent is checking your calendar

not:

```
AXUIElementPerformAction(kAXPressAction)
```

And:

Your help is needed to approve a security prompt

not:

```
kAXErrorAPIDisabled / secure desktop detected .
```

Primary sources, risks, and final decision

Sources to treat as authoritative during implementation

For **macOS**, the build specification should pin Apple documentation for `AXUIElement`, Accessibility/TCC permission behavior, ScreenCaptureKit, Developer ID signing/notarization, hardened-runtime requirements, and Virtualization.framework. ³⁵

For **Windows**, prioritize Microsoft's current WinApp UI Automation documentation, UIPI/UIAccess and UAC documentation, Windows Sandbox requirements, MSIX/App Installer distribution documentation, and code-signing/SmartScreen guidance. ³⁶

For **browsers**, pin Playwright's locator, page and CDP documentation. In particular, preserve the distinction between normal Playwright control and lower-fidelity attachment to arbitrary existing Chromium via CDP.

7

For **local inference**, prioritize the official Qwen model/release repositories, ByteDance's UI-TARS checkpoint/model card, `llama.cpp`, Apple's MLX/MLX-LM, Ollama's hardware documentation, vLLM/vLLM-Metal, and Google's Gemma 4 model card. 37

For **computer-use implementation references**, the highest-value repositories to audit now are Cua/Cua Driver, UI-TARS Desktop/Agent TARS, Agent S, Trope CUA, BrowserGym/OSWorld, Browser Use and OmniParser. Cua Driver is particularly important because it now implements many of the cross-platform background-control concepts that the original report proposed abstractly. 38

Assumptions that must be removed from the product specification

Assumption	Verdict	Replacement
"RTX 4070" means 12 GB VRAM	False for laptops	Probe actual memory; laptop 4070 is 8 GB while desktop is 12 GB. 20
Every Mac can run the same model	False	Hardware-adaptive model profiles
A 7B-9B VLM should always remain resident	Unreliable	swap/on-demand visual model on constrained tiers
vLLM is a natural Windows desktop runtime	False upstream	llama.cpp primary; vLLM benchmark via Linux/WSL or non-P0 paths. 39
AX/UIA gives universal arbitrary application control	False	capability discovery + fallback
Semantic automation always preserves focus	Unproven on macOS/app-specific	record and certify background-safe actions
Physical-coordinate host control can coexist invisibly with the human	Not a general solution	isolated environment or handoff
Every Windows machine can use Windows Sandbox	False	Pro/Enterprise/Education only; optional provider. 40
Every low-memory Mac should run a VM fallback	Bad product assumption	VM capability-gated; low tier hands off
Signing/notarization grants Accessibility/Screen Recording	False	user permission onboarding. 5
UAC/TCC/security prompts can be silently automated	Wrong product/security assumption	explicit human handoff

Assumption	Verdict	Replacement
CAPTCHA/MFA should be solved autonomously	Wrong boundary	human handoff
Existing personal Chrome can always be attached with full Playwright behavior	Unreliable	agent-managed profile default; CDP secondary. ⁴¹
A high OSWorld score proves the small local model works	False inference	benchmark the exact local configuration
Huge context solves multi-hour reliability	Rejected by baseline architecture	durable external state + bounded working context. <code>filecite{return0file0}</code>
One model should handle every hardware tier	Wrong product design	HardwareProfile → InferenceProfile
A VM can transparently access every host-native application	False product abstraction	VM is a separate provisioned workspace

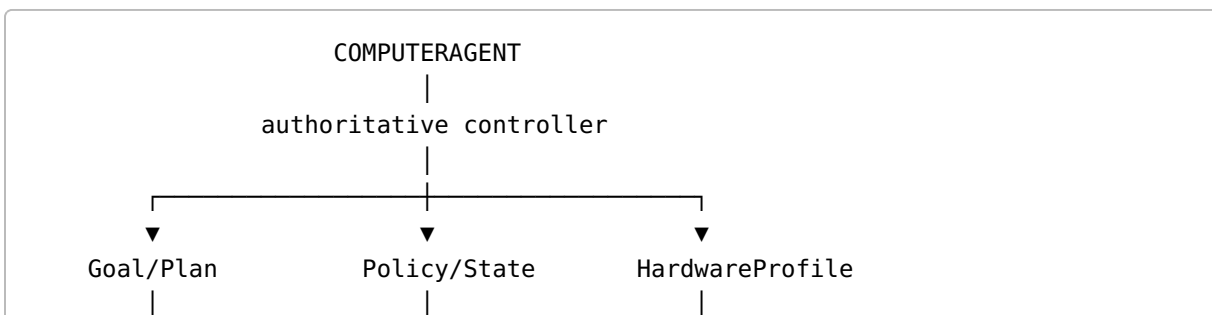
Final decision

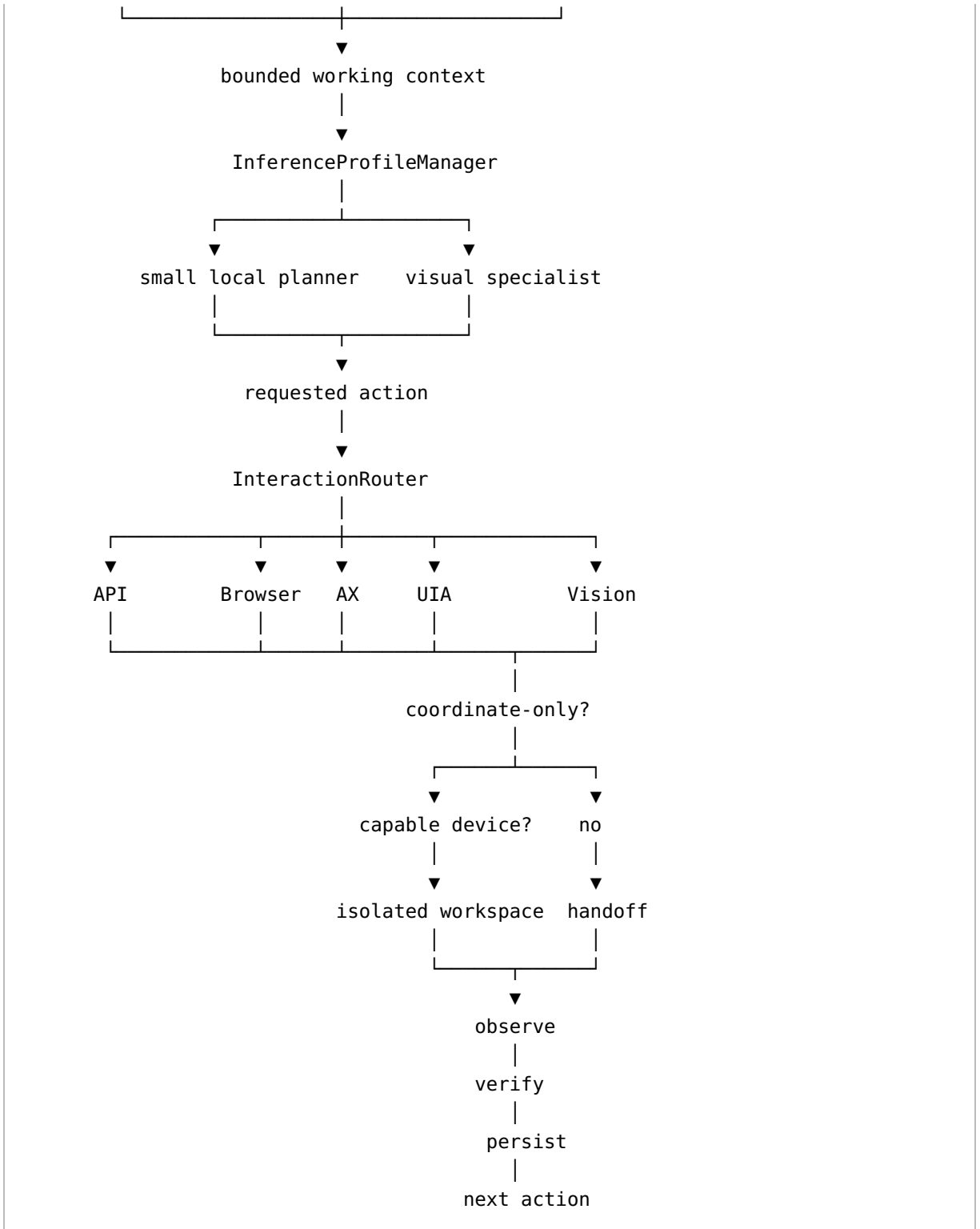
ComputerAgent should proceed, with macOS and Windows both promoted to P0.

The deepest architecture in the original report does not need to be replaced. It should be **extended in six specific ways**:

- `HardwareProfiler` becomes a first-class component.
- `InferenceProfileManager` selects models/runtimes based on measured resources.
- `PermissionBroker` explicitly models macOS TCC, Windows security boundaries and human handoff.
- Accessibility actions gain per-action **background-safety capability metadata**.
- `IsolatedDesktopProvider` becomes optional/capability-gated rather than a guaranteed universal fallback.
- Consumer packaging, model management, signing and update/recovery become architecture requirements rather than final-stage deployment work.

The resulting target is:





That is a realistic route to the product goal: **an installable ComputerAgent for other people's Macs and Windows laptops that runs locally, adapts to their hardware, works for long periods without using its prompt as memory, avoids stealing their physical input whenever the operating system provides a safe semantic path, and knows when to stop and ask for help.**

The Phase 0 experiments—not UI implementation—should now be the next coding milestone.

- 1 GeForce RTX 4070 Family Graphics Cards | NVIDIA
https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4070-family/?utm_source=chatgpt.com
- 2 9 36 UI Automation - Windows apps | Microsoft Learn
https://learn.microsoft.com/en-us/windows/apps/dev-tools/winapp-cli/ui-automation?utm_source=chatgpt.com
- 3 21 23 Virtualization | Apple Developer Documentation
https://developer.apple.com/documentation/virtualization?utm_source=chatgpt.com
- 4 35 AXUIElement.h | Apple Developer Documentation
https://developer.apple.com/documentation/applicationservices/axuielement_h?utm_source=chatgpt.com
- 5 34 Allow accessibility apps to access your Mac – Apple Support (UK)
https://support.apple.com/en-gb/guide/mac-help/mh43185/mac?utm_source=chatgpt.com
- 6 app | Electron
https://www.electronjs.org/docs/latest/api/app?utm_source=chatgpt.com
- 7 11 Locators | Playwright
https://playwright.dev/docs/locators?utm_source=chatgpt.com
- 8 12 24 38 cua/libs/cua-driver/rust/Skills/cua-driver/README.md at main · trycua/cua · GitHub
https://github.com/trycua/cua/blob/main/libs/cua-driver/rust/Skills/cua-driver/README.md?utm_source=chatgpt.com
- 10 User Account Control Allow UIAccess applications to prompt for elevation without using the secure desktop - Windows 10 | Microsoft Learn
https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/security-policy-settings/user-account-control-allow-uiaccess-applications-to-prompt-for-elevation-without-using-the-secure-desktop?utm_source=chatgpt.com
- 13 30 37 Qwen3.8/README.md at main · QwenLM/Qwen3.8 · GitHub
<https://github.com/QwenLM/Qwen3.8/blob/main/README.md>
- 14 model-00001-of-00007.safetensors · ByteDance-Seed/UI-TARS-1.5-7B at 3ed9359ba593fa0a5a001b8403d96daa19877b79
<https://huggingface.co/ByteDance-Seed/UI-TARS-1.5-7B/blob/3ed9359ba593fa0a5a001b8403d96daa19877b79/model-00001-of-00007.safetensors>
- 15 Gemma releases | Google AI for Developers
https://ai.google.dev/gemma/docs/releases?utm_source=chatgpt.com
- 16 llama.cpp/README.md at master · ggml-org/llama.cpp · GitHub
https://github.com/ggml-org/llama.cpp/blob/master/README.md?utm_source=chatgpt.com
- 17 mlx-lm/mlx_lm/BENCHMARKS.md at main · ml-explore/mlx-lm · GitHub
https://github.com/ml-explore/mlx-lm/blob/main/mlx_lm/BENCHMARKS.md
- 18 39 GPU - vLLM
https://docs.vllm.ai/en/latest/getting_started/installation/gpu/?utm_source=chatgpt.com
- 19 ollama/docs/gpu.mdx at main · ollama/ollama · GitHub
<https://github.com/ollama/ollama/blob/main/docs/gpu.mdx>

20 GeForce RTX 40 Series Gaming Laptops | NVIDIA

https://www.nvidia.com/en-us/geforce/laptops/40-series/?utm_source=chatgpt.com

22 Use and configure Windows Sandbox | Microsoft Learn

https://learn.microsoft.com/en-us/windows/security/application-security/application-isolation/windows-sandbox/windows-sandbox-configure-using-wsb-file?source=recommendations&utm_source=chatgpt.com

25 Signing Mac Software with Developer ID - Apple Developer

https://developer.apple.com/developer-id/?utm_source=chatgpt.com

26 Preparing your app for distribution | Apple Developer Documentation

https://developer.apple.com/documentation/xcode/preparing-your-app-for-distribution?utm_source=chatgpt.com

27 Choose a distribution path for your Windows app - Windows apps | Microsoft Learn

https://learn.microsoft.com/en-us/windows/apps/package-and-deploy/choose-distribution-path?utm_source=chatgpt.com

28 Code signing options for Windows app developers - Windows apps | Microsoft Learn

https://learn.microsoft.com/en-us/windows/apps/package-and-deploy/code-signing-options?utm_source=chatgpt.com

29 Auto-update and repair apps - MSIX | Microsoft Learn

https://learn.microsoft.com/en-us/windows/msix/app-installer/auto-update-and-repair--overview?utm_source=chatgpt.com

31 GitHub - tropeai/trope-cua: Background computer use for desktop agents on Windows and macOS · GitHub

https://github.com/tropeai/trope-cua?utm_source=chatgpt.com

32 GitHub - bytedance/UI-TARS-desktop: The Open-Source Multimodal AI Agent Stack: Connecting Cutting-Edge AI Models and Agent Infra · GitHub

https://github.com/bytedance/ui-tars-desktop?utm_source=chatgpt.com

33 Agent-S/README.md at main · similar-ai/Agent-S · GitHub

https://github.com/similar-ai/Agent-S/blob/main/README.md?utm_source=chatgpt.com

40 Windows-Sandbox | Microsoft Learn

https://learn.microsoft.com/de-de/windows/security/application-security/application-isolation/windows-sandbox/?utm_source=chatgpt.com

41 BrowserType | Playwright Python

https://playwright.dev/python/docs/api/class-browsertype?utm_source=chatgpt.com